AI-Assisted GitHub Development with IBM Bob, Langflow & MCP

Guideline Pengembangan Project — Local Laptop, Gemini Free Tier, GitHub

1. Keputusan Project yang Sudah Dikunci

Platform

Langflow OSS • berjalan lokal di laptop

AI / LLM

Google Gemini API • Free Tier

GitHub

GitHub.com • 1 public repository

Agent / IDE

IBM Bob sebagai coding assistant

Integration

Official GitHub MCP Server • tanpa Composio

Workflow

AI boleh write/commit → PR → human review → merge

2. Tujuan Project

Memungkinkan developer membangun dan mengubah AI workflow Langflow dengan bantuan IBM Bob.

Memungkinkan Agent di Langflow membaca dan mengubah repository GitHub melalui MCP.

Menjadikan GitHub sebagai source of truth untuk source code, konfigurasi, dan flow Langflow.

AI boleh membuat branch, mengubah file, commit, dan Pull Request.

Keputusan merge tetap berada pada manusia untuk menjaga kontrol dan keamanan.

3. Arsitektur Utama

Semua komponen inti berjalan lokal; GitHub diakses melalui HTTPS.



IBM Bob → membantu developer membuat/mengubah flow melalui Langflow MCP.

Langflow → menjadi orchestration layer dan tempat Agent dijalankan.

Gemini → menjadi LLM untuk reasoning dan pemilihan tool.

GitHub MCP → menjadi jembatan standar antara Agent dan GitHub.

GitHub → menyimpan repository dan riwayat perubahan.

4. Dua Jalur MCP yang Berbeda

MCP #1 — Bob → Langflow

Digunakan saat developer meminta Bob membantu:
- membuat flow
- mengubah flow
- debugging
- menjalankan/test flow

Bob berperan sebagai coding/developer assistant.

MCP #2 — Langflow → GitHub

Digunakan saat Agent Langflow perlu:
- membaca repository
- membaca file
- membuat branch
- mengubah file
- commit
- membuat Pull Request

GitHub MCP menjadi capability layer.

5. Peran Setiap Komponen

IBM Bob

Interface developer.

Membantu coding, membuat flow, melakukan perubahan, dan mengarahkan pekerjaan Langflow.

Langflow

Orchestration layer.

Menjalankan Agent, menghubungkan Gemini dengan tools, dan menyimpan flow.

Gemini

Otak Agent.

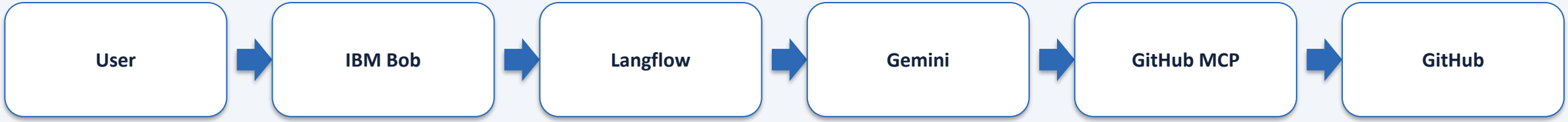
Memahami instruksi, melakukan reasoning, memilih tools, dan menyusun respons.

GitHub MCP

Jembatan ke GitHub.

Memberikan kemampuan repository, content, branch, commit, PR, dan tools lain yang diizinkan.

6. Workflow Utama Project



User memberikan tugas → Bob membantu mengembangkan atau menjalankan workflow.

Langflow menjalankan Agent → Gemini melakukan reasoning.

Agent memanggil GitHub MCP sesuai kebutuhan.

GitHub MCP menjalankan operasi terhadap repository.

Perubahan masuk ke branch AI → commit → Pull Request.

Developer melakukan review dan menentukan apakah perubahan di-merge.

7. Struktur Repository yang Disarankan

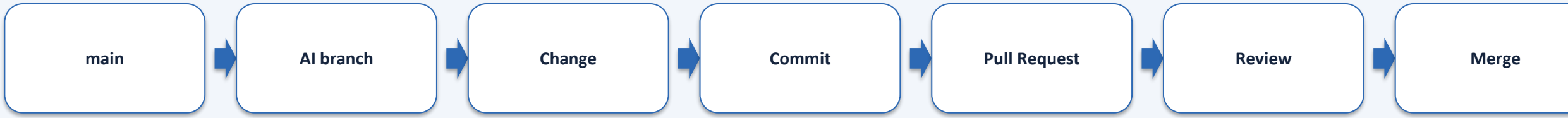
```
ai-github-langflow/
├── .bob/
│   └── mcp.json
├── langflow/
│   ├── flows/
│   │   └── github-agent.json
│   └── README.md
├── docs/
│   ├── architecture.md
│   ├── setup.md
│   ├── security.md
│   └── workflow.md
├── github-mcp/
│   └── README.md
├── .env.example
├── .gitignore
├── docker-compose.yml
├── README.md
└── LICENSE
```

Flow Langflow disimpan di GitHub agar dapat version control.

Secret aktual tidak boleh masuk repository/public.

Dokumentasi teknis disimpan bersama project.

8. Git & Pull Request Strategy



AI tidak bekerja langsung pada main/default branch.

Branch contoh: ai/fix-langflow-agent atau ai/update-flow.

Setiap perubahan memiliki commit message yang jelas.

Pull Request menjadi titik kontrol sebelum perubahan masuk main.

Merge tetap dilakukan oleh manusia.

9. Kemampuan Agent pada MVP

Read

- melihat repository
- membaca file
- mencari kode
- memahami flow

Write

- membuat branch
- mengubah file
- memperbarui flow
- membuat commit

Pull Request

- membuat PR
- membaca PR
- merangkum perubahan
- membantu review

Phase berikutnya

Issues, GitHub Actions, security/code quality, dan audit dapat ditambahkan setelah MVP stabil.

10. Toolset GitHub MCP — Bertahap

MVP

context
repos
pull_requests

Fokus: repository, code, branch, PR.

Phase 2

issues
actions

Fokus: issue tracking dan CI/CD.

Phase 3

code_security
code_quality
dependabot

Tambahkan sesuai kebutuhan.

Jangan expose seluruh tool sejak awal. Gunakan prinsip least privilege.

Semakin sedikit tool, semakin kecil context dan surface area risiko.

11. Keamanan Credential

Benar

`.env` / secret manager

GITHUB_PERSONAL_ACCESS_TOKEN=...
GEMINI_API_KEY=...
LANGFLOW_API_KEY=...

`.env` masuk `.gitignore`.

Salah

Jangan menaruh API key di:
- source code
- README
- flow JSON
- `.bob/mcp.json` sebagai hardcode
- commit history
- repository public

12. GitHub Authentication untuk MVP

Gunakan Fine-grained Personal Access Token karena project bersifat personal, satu repository, dan local.

Batasi token hanya ke repository yang diperlukan.

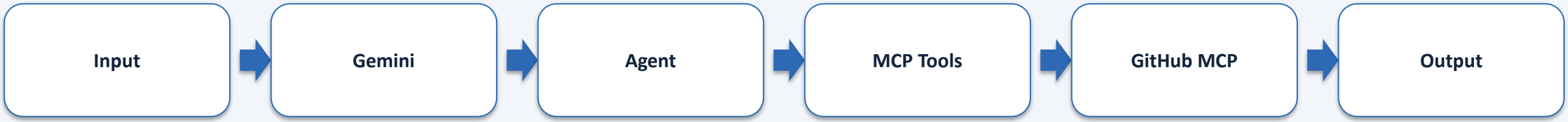
Repository permission minimum yang direkomendasikan:

- Contents → Read and write
- Pull requests → Read and write
- Metadata → Read

Tambahkan Issues → Read and write hanya jika memang diperlukan.

Jangan berikan permission merge/admin bila tidak dibutuhkan.

13. Desain Flow Langflow MVP



Input menerima tugas developer.

Gemini memberikan reasoning dan menentukan kapan tools perlu dipanggil.

Agent mengakses MCP Tools.

MCP Tools meneruskan operasi ke GitHub MCP Server.

Tool result kembali ke Agent → Gemini → output akhir.

14. Contoh Skenario End-to-End

Developer: “Periksa flow GitHub Agent. Jika ada masalah, perbaiki dan buat Pull Request.”

1. Agent membaca struktur repository.
2. Agent menemukan file flow Langflow.
3. Agent membaca dan menganalisis flow.
4. Agent membuat branch: ai/fix-github-agent.
5. Agent mengubah file yang bermasalah.
6. Agent melakukan validasi/test yang tersedia.
7. Agent membuat commit.
8. Agent membuat Pull Request ke main.
9. Developer melakukan review → approve/reject → merge jika sesuai.

15. Environment Local

Laptop / Windows

IBM Bob
Git
Docker Desktop
Browser

Semua komponen development utama berjalan lokal.

Docker

Langflow container
GitHub MCP container

Docker membantu menjaga environment konsisten.

Internet

Gemini API
GitHub.com

Koneksi keluar menggunakan HTTPS.

16. Database: Belum Diperlukan pada MVP

GitHub menjadi source of truth untuk source code, flow, branch, commit, dan Pull Request.

Langflow menangani execution dan workflow.

Karena project masih personal dan local, PostgreSQL belum diperlukan.

Database dapat ditambahkan nanti untuk audit log, execution history, analytics, approval tracking, atau multi-user.

17. Risiko Keamanan Utama

Credential Leak

API key / GitHub token masuk repository public.

Mitigasi: ENV, .gitignore, least privilege.

Prompt Injection

README, issue, PR, atau source code berisi instruksi jahat.

Mitigasi: treat repository content as untrusted data.

Bad Change

AI membuat perubahan yang merusak aplikasi.

Mitigasi: branch + commit + PR + human review.

Over-permission

Tool/credential memiliki akses terlalu luas.

Mitigasi: tool allow-list + least privilege.

18. Roadmap Implementasi

1. Environment

Install Bob, Git, Docker, Langflow

2. GitHub

Repository + fine-grained PAT

3. GitHub MCP

Run + test authentication

4. Langflow

Gemini + MCP Tools

5. Agent

Read repository + tool calling

6. Write

Branch + modify + commit

7. PR

Create PR + human review

19. Urutan Pengembangan yang Aman

1. Jalankan Langflow dan pastikan UI/API berfungsi.
2. Hubungkan Gemini dan tes Agent tanpa GitHub.
3. Jalankan GitHub MCP dan tes akses repository.
4. Hubungkan Langflow → GitHub MCP.
5. Tes read-only: repository, file, branch.
6. Aktifkan write: branch → update file → commit.
7. Tes create Pull Request.
8. Hubungkan IBM Bob → Langflow.
9. Simpan flow final ke GitHub.
10. Dokumentasikan setup dan security.

20. Definition of Done — MVP

- ☐ Bob dapat terhubung ke Langflow.
- ☐ Langflow dapat menggunakan Gemini.
- ☐ Langflow Agent dapat terhubung ke GitHub MCP.
- ☐ Agent dapat membaca repository dan file.
- ☐ Agent dapat membuat branch.
- ☐ Agent dapat mengubah file dan commit.
- ☐ Agent dapat membuat Pull Request.
- ☐ Flow Langflow tersimpan di GitHub.
- ☐ Secret tidak masuk repository.
- ☐ Merge tetap dikontrol manusia.
- ☐ Dokumentasi setup tersedia.

21. Pengembangan Setelah MVP

CI/CD

GitHub Actions
Automated tests
Linting
Validation

Security

GitHub App
Secret manager
Audit log
Prompt-injection defenses

Scale

Private repos
Multiple repositories
Team users
PostgreSQL

Prinsip

Jangan menambah kompleksitas sebelum alur dasar stabil dan dapat diuji end-to-end.

22. Target Akhir Project



Developer cukup memberikan instruksi tingkat tinggi.

AI memahami repository dan flow yang relevan.

AI melakukan perubahan secara terkontrol.

Setiap perubahan terdokumentasi melalui Git.

Pull Request menjadi checkpoint sebelum merge.

Developer tetap memegang keputusan final.

23. Checklist Hari Pertama

☐ Buat repository GitHub public.

☐ Buat struktur folder project.

☐ Install Git.

☐ Install Docker Desktop.

☐ Pastikan IBM Bob siap digunakan.

☐ Jalankan Langflow lokal.

☐ Siapkan Gemini API key.

☐ Buat fine-grained GitHub PAT.

☐ Jalankan official GitHub MCP Server.

☐ Jangan commit `.env` atau token.

☐ Setelah semua siap, lakukan test koneksi satu per satu.

Baseline Project Architecture

IBM Bob + Langflow OSS + Gemini + GitHub MCP + GitHub

Mulai dari koneksi sederhana → read → write → commit → PR → human review